

CLINICAL AI CONTROL PLANE

Enterprise Standard

CAG-04 · Coding and Architecture Standard

Prepared by Engineering · 2026-08-21

Clinical AI Platform Group

PHI PROTECTION · FULL AUDIT TRAIL · HIPAA ALIGNED

Clinical AI Control Plane: Enterprise Coding and Architecture Standard

The coding contract the implementation conforms to and the reference the architecture spec builds on

SECTION 01

Layered infrastructure (strict, one direction of dependency)

```
HTTP -> Controller (thin) -> Service (orchestration) -> Repository -> PostgreSQL
             input DTO / validator        |                (tenant-filtered reads
             <- typed response DTO <-     +-> ModelProvider | RedactionService |
                                              UsageMeteringService | GovernancePo
                                              AuditPublisher
```

- **Controller (thin):** parse the request into a validated DTO, extract tenant and actor from the verified JWT, call the service, return the response DTO. No query building, no persistence, no inline policy logic.
- **Service (single orchestrator):** `SuggestionService` owns the governed lifecycle: meter, redact, call the provider, persist, evaluate policy, audit. Domain rules that belong to the entity stay in the entity.
- **Repository:** Spring Data interfaces only. Every read that returns tenant data takes the tenant id in the query (`findByIdAndTenantId` , `findByTenantIdOrderByCreatedAtDesc`). No post-load filtering.
- **DTO boundary:** entities never cross the HTTP boundary. Request records (`CreateSuggestionRequest` , `DecisionRequest`) validate with Jakarta annotations; response records (`SuggestionResponse` , `AuditEventResponse`) shape the contract.

SECTION 02

Configuration over code

- Every governance behavior is externalized under a typed `app.*` namespace in `application.yml` with environment-variable bindings and safe defaults: `app.model.provider`, `app.usage.mode`, `app.usage.requests-per-minute`, `app.governance.review-required-assistants`, `app.kafka.audit-topic`.
- Changing where autonomy applies, which model backend runs, or whether budgets enforce is a deployment change, never a code change.
- No magic strings on the configuration path; domain vocabulary is typed (`AssistantType`, `SuggestionStatus`, `GroundingSourceType`, `AuditEventType` enums). An invalid assistant type fails validation at the boundary, not silently downstream.

SECTION 03

Security first (non-negotiable in this compliance environment)

- **Stateless resource server:** every `/api` request carries a verified JWT; sessions and CSRF are disabled because there is no state to protect. Only health probes, the OpenAPI UI, and the dev token endpoint are open.
- **Tenant scope in the query, resolved once:** the `tenant_id` claim is mandatory and applied at the repository query. A missing claim is an error, never a silent fallback.
- **Scope-gated writes:** create, decide (approve/reject), and execute are separate authorities enforced with method-level `@PreAuthorize`, so a reviewer credential cannot create and a creator credential cannot approve.
- **Data minimization before the model:** PHI redaction runs before any provider call, always on, not configurable off. Provider contracts never determine what leaves the trust boundary.
- **Read and write audit:** every lifecycle transition emits an audit event with tenant, actor, and detail; the trail is persisted by an independent consumer and queryable per tenant and per suggestion.
- **Structured errors, never silent skips:** invalid transitions throw, unknown ids return a typed 404, budget rejections return a typed 429; all through one `GlobalExceptionHandler` returning a structured `ApiError`.
- **Secrets posture:** the only key in the repository is a throwaway dev signing key, documented as such; production keys come from the identity provider and a managed secret store. CI runs a Trivy scan for vulnerabilities, secrets, and misconfigurations.

SECTION 04

Enterprise code quality

- **Typed everything:** records for DTOs and value objects, enums for states and types, interfaces for swappability (`ModelProvider`). The state machine lives in the `Suggestion` entity and throws on invalid transitions.
- **Schema by migration only:** Flyway owns the schema (`V1__init.sql`); Hibernate runs with `ddl-auto: none` . A schema change is a new versioned migration, reviewed like code.
- **First-party, supported components** on the request path: Spring Web, Spring Security, Spring Data JPA, Spring Kafka, Spring Data Redis. No experimental drivers.
- **Formatting as a gate:** google-java-format via Spotless; `spotlessCheck` runs as its own CI job so a fresh clone builds green, and `spotlessApply` runs before commit.
- **Tests at two altitudes:** unit tests (Mockito) for redaction, metering in both modes, and orchestration on both policy paths; one integration test (Testcontainers: Postgres, Kafka, Redis) driving the real HTTP flow with a signed token and awaiting the audit consumer.
- **Contract first:** the API is described by springdoc OpenAPI and served from the running application, so documentation cannot drift from the implementation.
- **Observability:** actuator health probes wired to Kubernetes liveness and readiness; per-tenant usage counters exposed over the API as the cost signal.

SECTION 05

Anti-pattern replacements

COMMON DEFECT	STANDARD IN THIS CODEBASE
Assistants call the model vendor directly	All model access through the control plane's `ModelProvider` seam
PHI handling delegated to provider contracts	Redaction at the gateway boundary, always on, before any call
Review rules hardcoded per feature	`GovernancePolicy` driven by configuration; autonomy by default
Hard rate limits as the first cost control	Metering and soft budgets; enforcement is an explicit opt-in mode
Audit as scattered log statements	Typed `AuditEvent` stream on Kafka with a persistent read model
Status changed by setters from anywhere	State machine methods on the entity; invalid transitions throw
Tenant filtering in application code after load	Tenant id bound into every repository query
Hibernate auto-DDL mutating production schema	Flyway migrations only, `ddl-auto: none`
Entities serialized straight to JSON	Request and response DTOs at the HTTP boundary
Formatting debates in review	google-java-format enforced by a CI gate

This standard is the coding contract for the control plane implementation and the reference the architecture specification (CAG-01) builds on.